



Memoria del Proyecto

SmartWorks — Gestión de pedidos, stock y reportes para una empresa de palets de fruta

Grado Superior DAM (2º) · 17/03/2026

Tecnologías	Spring Boot 3 (Java 21), MySQL, Flutter (Android)
Módulos backend	Productos, Clientes, Pedidos, Movimientos, Dashboard, Analítica mensual, Reportes PDF
Objetivo	Centralizar el stock de palets y los pedidos, con indicadores y reportes descargables

Índice

1. Introducción y contexto
2. Objetivos
3. Alcance y requisitos
4. Arquitectura y tecnologías
5. Modelo de datos
6. Diseño de la API REST
7. Analítica y reportes en PDF
8. Aplicación Flutter (estructura inicial)
9. Seguridad (modo dev) y buenas prácticas
10. Pruebas realizadas
11. Plan de trabajo y mejoras futuras
12. Conclusiones

1. Introducción y contexto

SmartWorks es una aplicación orientada a una empresa que trabaja con **palets de fruta**. El objetivo es controlar el **stock**, registrar **pedidos** de clientes y disponer de **indicadores** (analítica) para el seguimiento mensual, incluyendo la generación de **reportes en PDF**.

El proyecto está dividido en dos partes: **backend** (Spring Boot + MySQL) y **frontend móvil** (Flutter). En esta memoria se describe lo implementado, las decisiones técnicas y los flujos principales.

2. Objetivos

Objetivo general: construir un sistema sencillo y usable para gestionar pedidos y stock con reportes.

Objetivos específicos:

- Crear una API REST con endpoints claros para productos, clientes y pedidos.
- Validar stock en la creación de pedidos y actualizarlo en transacciones.
- Calcular analítica mensual: pedidos, estado, unidades vendidas, ingresos y media de entrega.
- Generar un PDF mensual descargable con el resumen de KPIs y top productos.
- Conectar Flutter a la API y mostrar dashboards/listados con edición básica.

3. Alcance y requisitos

Incluido:

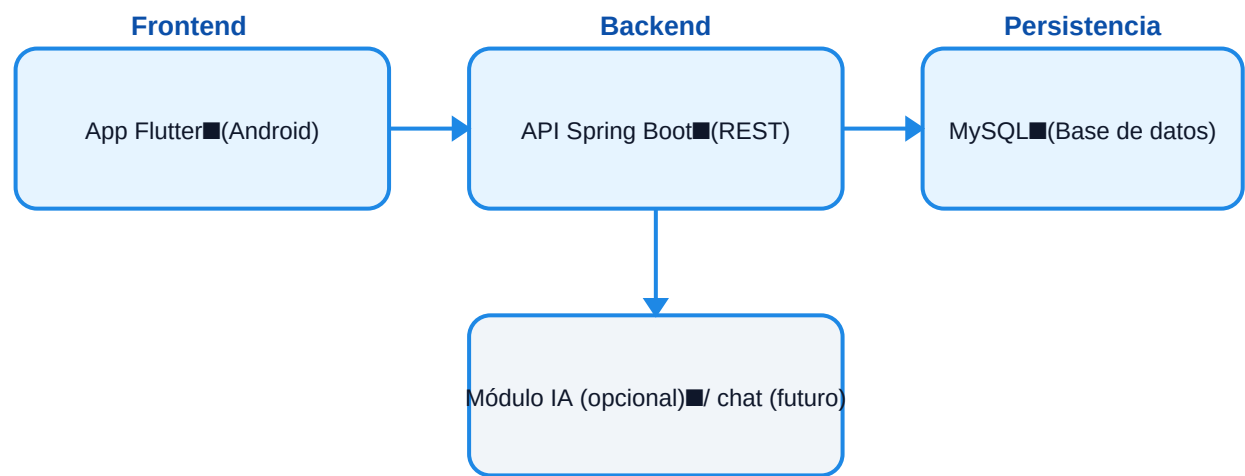
- CRUD de Productos (incluye precio, stock y umbral de stock bajo).
- CRUD de Clientes.
- Creación de Pedidos con líneas (OrderLine) y validación de stock.
- Dashboard rápido con contador de productos en stock bajo y pedidos pendientes.
- Analítica mensual y generación/descarga de reportes PDF.

No incluido todavía (futuro):

- Chatbot con acciones (consultas de stock/pedidos desde lenguaje natural).
- Roles/usuarios y seguridad completa en producción.
- Reportes avanzados (gráficas embebidas, histórico por meses, export CSV).

4. Arquitectura y tecnologías

La arquitectura es cliente-servidor: la app Flutter consume la API REST de Spring Boot. La persistencia se realiza en MySQL mediante JPA/Hibernate.



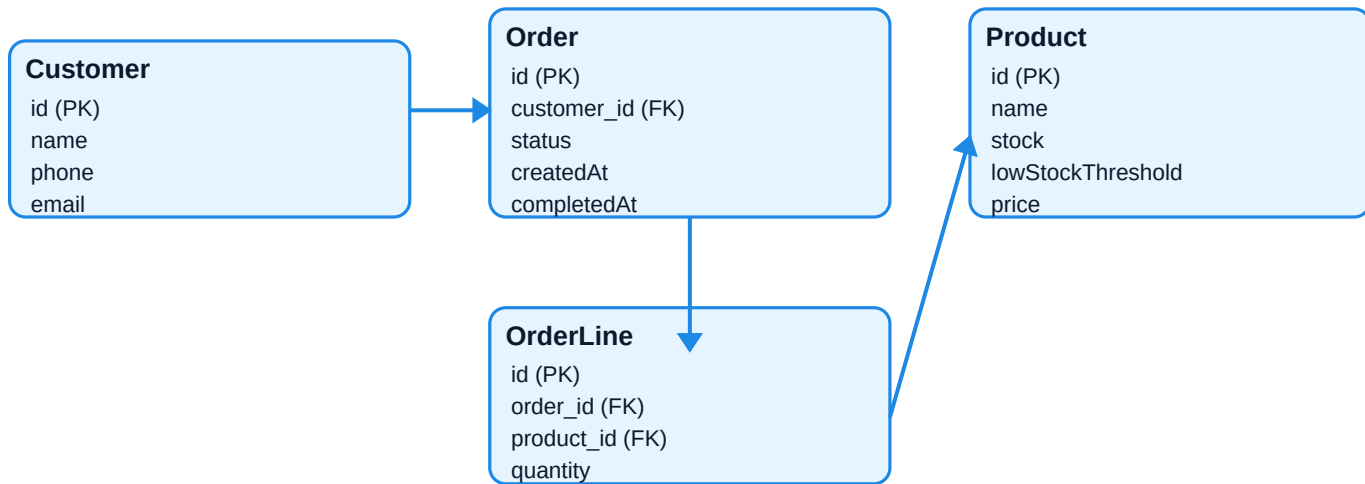
Principales tecnologías:

Backend	Java 21 · Spring Boot 3 · Spring Data JPA · (Security en modo dev)
Base de datos	MySQL 8
PDF	Apache PDFBox para reportes mensuales
Frontend	Flutter (Material 3) + consumo HTTP
Testing manual	Postman para probar endpoints

5. Modelo de datos

Las entidades principales son: **Product**, **Customer**, **Order** y **OrderLine**. Order tiene una relación 1:N con OrderLine. Cada OrderLine referencia un Product.

Modelo de datos (ER) simplificado



Notas de implementación:

- Order.status permite estados PENDING, COMPLETED y CANCELLED.
- Order.completedAt se usa para calcular la media de tiempo de entrega (horas).
- Product.lowStockThreshold permite alertas de stock bajo.

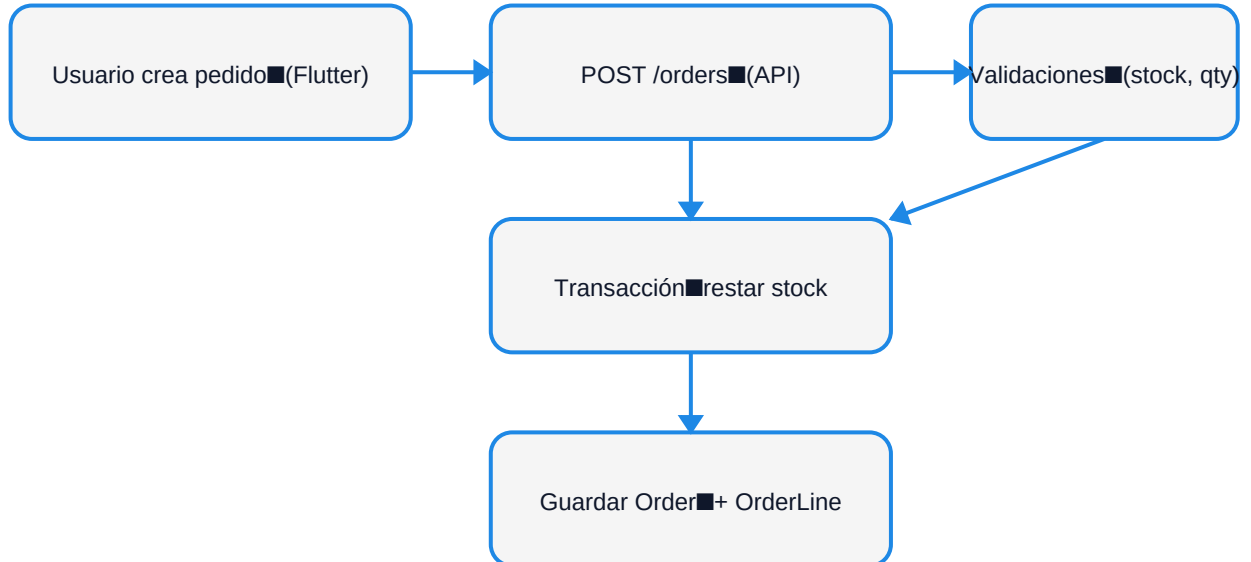
6. Diseño de la API REST

Se definieron endpoints REST sencillos, pensados para que Flutter consuma fácilmente listados y acciones. A continuación se muestra un resumen (puede variar según el proyecto final).

Recurso	Método	Ruta	Descripción
Productos	GET	/products	Lista productos (con filtro opcional stock bajo)
Productos	PATCH	/products/{id}	Editar nombre/stock/umbral/precio
Clientes	GET	/customers	Lista clientes
Clientes	POST	/customers	Crear cliente
Pedidos	GET	/orders	Lista pedidos
Pedidos	POST	/orders	Crear pedido con líneas y validación de stock
Pedidos	PATCH	/orders/{id}/status	Cambiar estado (y completedAt si aplica)
Dashboard	GET	/dashboard	KPIs rápidos: stock bajo + pedidos pendientes
Analytics	GET	/analytics/monthly	KPIs del mes (pedidos, ingresos, top productos, etc.)
Reports	POST	/reports/monthly	Genera y guarda un reporte PDF del mes
Reports	GET	/reports/{id}/download	Descarga el PDF generado

Flujo de creación de pedido:

Flujo: crear pedido



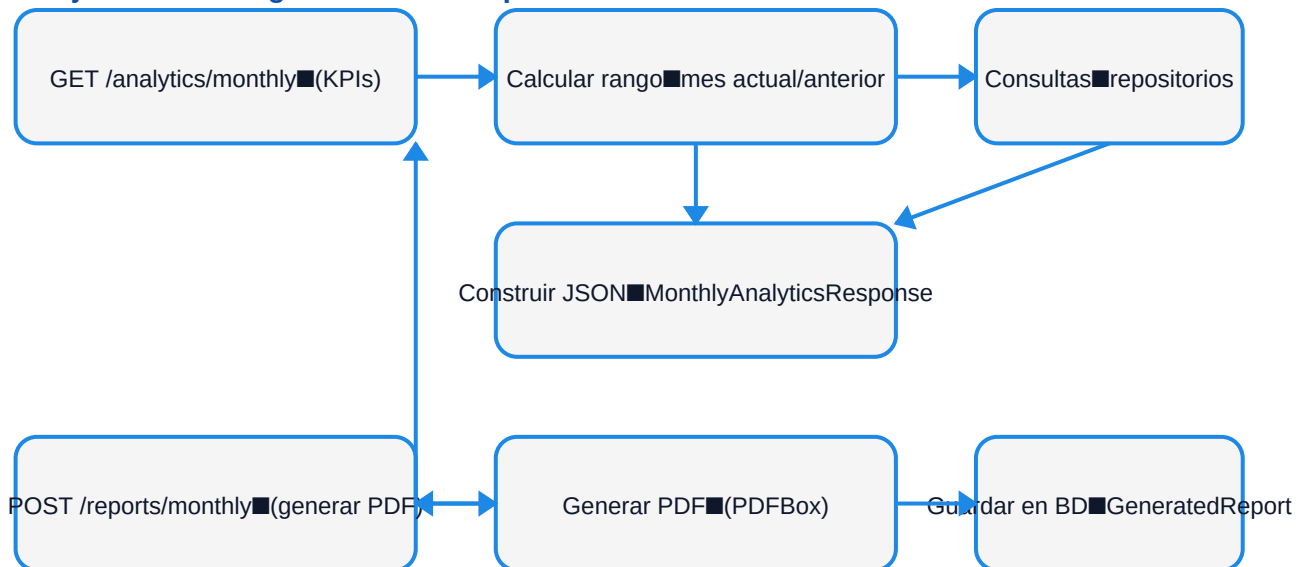
7. Analítica y reportes en PDF

El módulo de analítica calcula estadísticas mensuales usando el rango [inicio del mes, inicio del siguiente mes). Además se calcula el mes anterior para obtener porcentajes de cambio.

KPIs calculados (ejemplos):

- Pedidos del mes / mes anterior y % de variación.
- Pedidos por estado (PENDING/COMPLETED/CANCELLED).
- Unidades vendidas y top productos por unidades.
- Ingresos del mes (sumatorio quantity * price).
- Valor medio del pedido y media de horas hasta completado (si completedAt está presente).

Flujo: analítica + generación de reportes



Los reportes PDF se generan en el backend para asegurar consistencia y que el documento sea descargable desde Flutter (o desde Postman). El PDF se guarda asociado a un registro (GeneratedReport) en base de datos.

8. Aplicación Flutter (estructura inicial)

La app Flutter se organiza por pantallas (screens) y un ApiClient encargado de consumir la API. Actualmente se han implementado pantallas para: Dashboard, Productos, Pedidos (con creación) y Clientes, además de una navegación inferior (BottomNavigation).

Puntos clave de UI:

- Tema basado en tonos azul cielo coherentes con el logo.
- Listados con búsqueda, refresh (pull-to-refresh) y tarjetas (cards) para un look moderno.
- Productos con imágenes desde assets y detalle al pulsar.
- Formulario de nuevo pedido con selector de cliente y líneas dinámicas.

Navegación principal (ejemplo)



9. Seguridad (modo dev) y buenas prácticas

Durante el desarrollo se ha utilizado una configuración de seguridad simplificada para evitar bloqueos en pruebas (permitAll en entornos de desarrollo). La idea es reactivar JWT/roles en la fase final o para producción.

Buenas prácticas aplicadas:

- Transacciones en la creación de pedidos para evitar inconsistencias en stock.
- Validación de cantidades y control de stock insuficiente.
- Separación por capas: Controller → Service → Repository.
- Respuestas en JSON consistentes para consumo desde Flutter.

10. Pruebas realizadas

Las pruebas principales se realizaron con Postman y con la app Flutter conectada al backend local (localhost/10.0.2.2). Se validaron los casos habituales y algunos casos límite.

- Crear productos y editar stock/precio.
- Crear cliente y listar clientes.
- Crear pedido con stock suficiente (OK) y con stock insuficiente (error).
- Cambiar estado de pedido a COMPLETED y verificar completedAt para analítica.
- Consultar /analytics/monthly y comparar mes actual vs anterior.
- Generar PDF mensual y descargarlo desde endpoint /reports/{id}/download.

11. Plan de trabajo y mejoras futuras

Una vez estabilizada la API y la UI base, las siguientes mejoras aportan valor real al proyecto:

- **Chatbot con acciones:** sugerencias, búsquedas (stock bajo), creación de pedido guiada, descarga de reporte.
- **Histórico de reportes:** lista de reportes generados por mes y acceso desde la app.
- **Roles:** admin/empleador con permisos; reactivar JWT en producción.
- **Gráficas** en el analizador (por ejemplo, donut de estados y barras de top productos).
- **Optimización:** paginación de listados si crece el volumen de datos.

12. Conclusiones

El sistema SmartWorks cumple los objetivos de DAM: integra backend + base de datos + app móvil, aplicando un diseño por capas y proporcionando funcionalidad real para una empresa de palets de fruta. La analítica mensual y los reportes PDF aportan un elemento diferencial y justifican el módulo de analizador. Quedan extensiones futuras (chatbot, roles) que se pueden añadir sin rehacer la arquitectura.

Fin de la memoria.